

1. Given a weighted, directed graph $G = (V, E)$ with no negative-weight cycles, let m be the maximum over all vertices $v \in V$ of the minimum number of edges in a shortest path from the source s to v . (Here, the shortest path is by weight, not the number of edges.) Suggest a simple change to the Bellman-Ford algorithm that allows it to terminate in $m + 1$ passes, even if m is not known in advance.

Solution:

If the greatest number of edges on any shortest path from the source is m , then the path-relaxation property tells us that after m iterations of **BELLMAN-FORD**, every vertex v has achieved its shortest-path weight in $v.d$. By the upper-bound property, after m iterations, no d values will ever change. Therefore, no d values will change in the $(m + 1)$ st iteration. Because we do not know m in advance, we cannot make the algorithm iterate exactly m times and then terminate. But if the algorithm just stops when nothing changes any more, it will stop after $m + 1$ iterations.

BELLMAN-FORD-EARLY-TERMINATION(G, w, s)

INITIALIZE-SINGLE-SOURCE(G, s)

repeat

$changes = \text{FALSE}$

 for each edge $(u, v) \in G.E$

 if **RELAX'**(u, v, w)

$changes = \text{TRUE}$

until $changes == \text{FALSE}$

RELAX'(u, v, w)

 if $v.d > u.d + w(u, v)$

$v.d = u.d + w(u, v)$

$v.\pi = u$

 return TRUE

 else return FALSE

Because the exercise specifies that G has no negative-weight cycles, the test for a negative-weight cycle (based on there being a d value that would change if another relaxation step was done) has been removed. If there were a negative-weight cycle, this version of the algorithm would never get out of the repeat loop because some d value would change in each iteration.

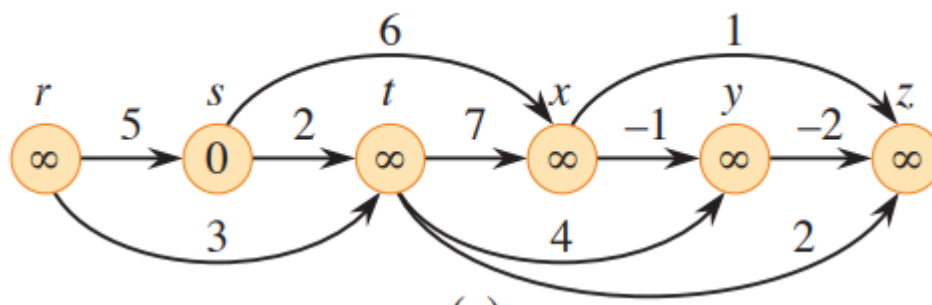
2. The following algorithm finds single source shortest paths in DAG. Dry run the following algorithm for finding shortest paths to all nodes, for the following graph, start from node r . Analyze the algorithm and provide its complexity in big O notations.

DAG-SHORTEST-PATHS(G, w, s)

```

1  topologically sort the vertices of  $G$ 
2  INITIALIZE-SINGLE-SOURCE( $G, s$ )
3  for each vertex  $u \in G.V$ , taken in topologically sorted order
4      for each vertex  $v$  in  $G.Adj[u]$ 
5          RELAX( $u, v, w$ )

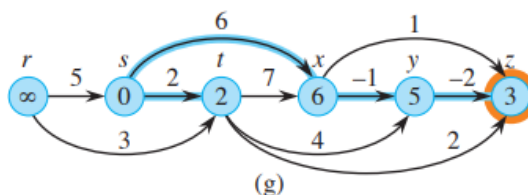
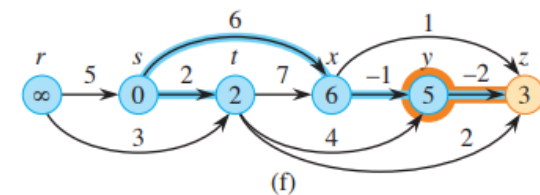
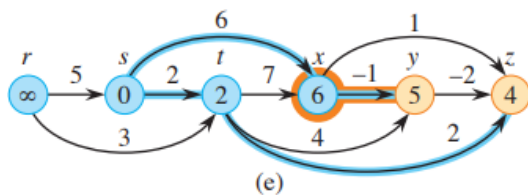
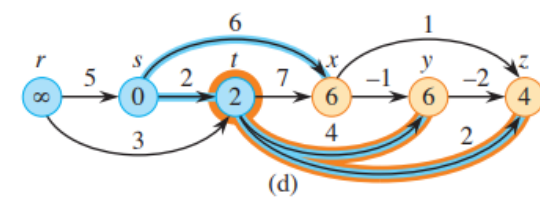
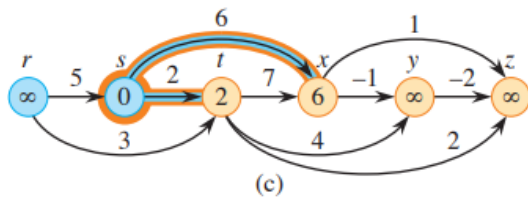
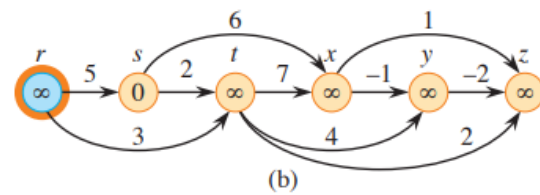
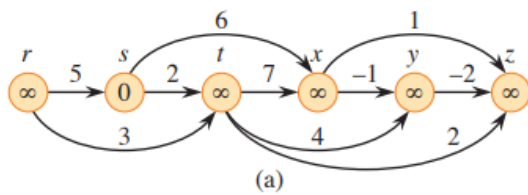
```



Submission not allowed afterwards

Total Marks: 100

Solution:



Let's analyze the running time of this algorithm. The topological sort of line 1 takes $\theta(V + E)$ time. The call of INITIALIZE-SINGLE-SOURCE in line 2 takes $\theta(V)$ time. The for loop of lines 3-5 makes one iteration per vertex. Altogether, the for loop of lines 4-5 relaxes each edge exactly once. (We have used an aggregate analysis here.) Because each iteration of the inner for loop takes $\theta(1)$ time, the total running time is $\theta(V + E)$ which is linear in the size of an adjacency-list representation of the graph.

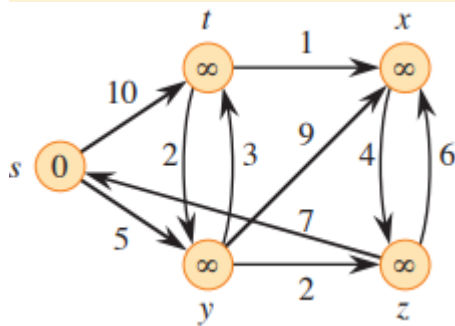
- Run Dijkstra's algorithm on the directed graph given below, using vertex s as the source and then using vertex z as the source. Show the d and π values and the vertices in set S after each iteration of the while loop.

DIJKSTRA(G, w, s)

```

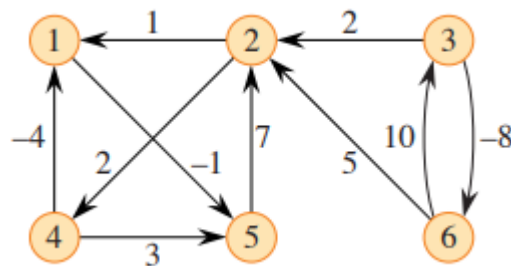
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = \emptyset$ 
4  for each vertex  $u \in G.V$ 
5      INSERT( $Q, u$ )
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8       $S = S \cup \{u\}$ 
9      for each vertex  $v$  in  $G.Adj[u]$ 
10         RELAX( $u, v, w$ )
11         if the call of RELAX decreased  $v.d$ 
12             DECREASE-KEY( $Q, v, v.d$ )

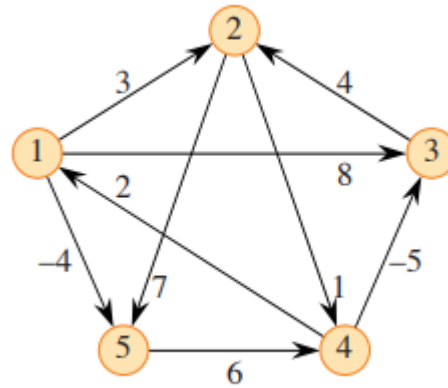
```



Solution:

4. Use Floyd-Warshall Algorithm to find all pair shortest paths in the following graphs. Specify all the path.





Solution:

$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(0)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \text{NIL} & 4 & \text{NIL} & \text{NIL} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(1)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(2)} = \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(3)} = \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad \Pi^{(4)} = \begin{pmatrix} \text{NIL} & 1 & 4 & 2 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(5)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad \Pi^{(5)} = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

Submission not allowed afterwards

Total Marks: 100

5. a) Conduct a DFS for the following graph. (Label each vertex u with the start time and the finish time) or (Show all steps showing stack and Visit Order). You should start the traversal from vertex B, and follow the alphabetic order whenever you need to make choices. [10 Points]

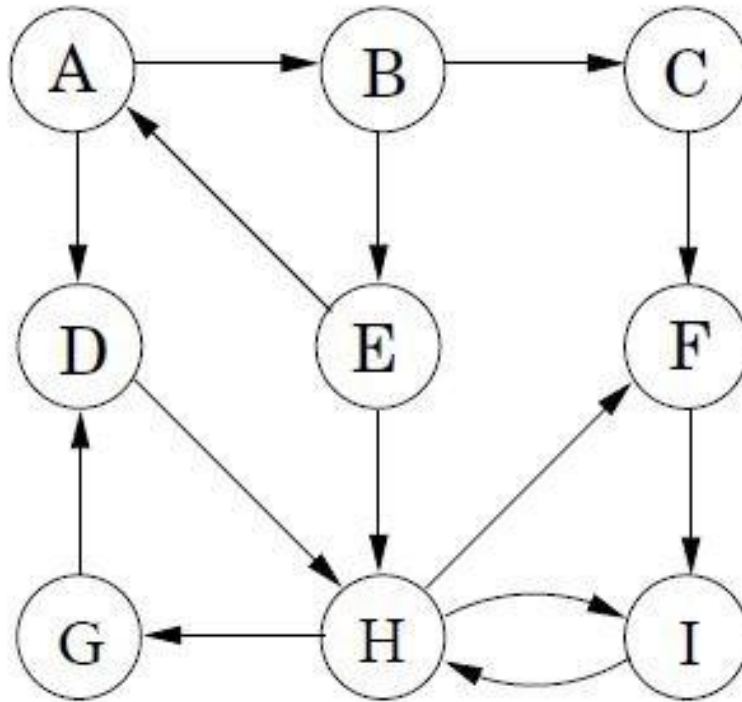


Figure 1: Graph for the DFS and SCC

Solution:**(a) DFS — discovery (start) and finish times**

I used standard DFS time increments (time starts at 1 and increments at each discovery and each finish).

Vertex	start (d)	finish (f)
B	1	18
C	2	13
F	3	12
H	4	11
G	5	8

Due Date: 28 November 2025

20% penalty for 1 day late

40% penalty for 2 days late

CS2009: Design and Analysis of Algorithms (Fall 2025)

Assignment 4

Submission not allowed afterwards

Total Marks: 100

D	6	7
I	9	10
E	14	17
A	15	16

Discovery order (when vertices are first visited):

$B \rightarrow C \rightarrow F \rightarrow H \rightarrow G \rightarrow D \rightarrow I \rightarrow (\text{backtrack}) \rightarrow (\text{backtrack}) \rightarrow E \rightarrow A$

(That is: B(1) discovers C(2), C discovers F(3), F discovers H(4), H discovers G(5), G discovers D(6) — then backtracking and later H discovers I(9). After finishing that subtree we return and from B visit E(14) then A(15).)

b) List all edges (back edges, forward edges, cross edges) that belong to each of the following sets: [5 Points]

List of directed edges (as read from the diagram) and their classification under this DFS:

Edges ($u \rightarrow v$) and class:

Tree edges (edges used first to discover a vertex):

- $B \rightarrow C$ (tree)
- $B \rightarrow E$ (tree)
- $C \rightarrow F$ (tree)
- $F \rightarrow H$ (tree)
- $H \rightarrow G$ (tree)
- $H \rightarrow I$ (tree)
- $G \rightarrow D$ (tree)
- $E \rightarrow A$ (tree)

Back edges (point to an ancestor in the DFS tree):

- $A \rightarrow B$ (back) — A points to ancestor B.
- $D \rightarrow H$ (back) — D points back to ancestor H (completing cycle $H \rightarrow G \rightarrow D \rightarrow H$).
- $I \rightarrow H$ (back) — I points back to ancestor H (makes H and I mutually reachable).

Forward edges (point to a descendant but are not tree edges):

- $F \rightarrow I$ (forward) — I is a descendant of F (discovered later under H), but $F \rightarrow I$ is not the tree edge ($H \rightarrow I$ is the tree edge), so this is a forward edge.

Cross edges (go between nodes in different DFS subtrees, neither ancestor nor descendant):

Submission not allowed afterwards

Total Marks: 100

- $A \rightarrow D$ (cross) — A points into a different, already finished subtree.
- $E \rightarrow H$ (cross) — $E \rightarrow H$ goes into another part already explored in this run.

(If you prefer, the complete list together:

- Tree: $\{B \rightarrow C, B \rightarrow E, C \rightarrow F, F \rightarrow H, H \rightarrow G, H \rightarrow I, G \rightarrow D, E \rightarrow A\}$
- Back: $\{A \rightarrow B, D \rightarrow H, I \rightarrow H\}$
- Forward: $\{F \rightarrow I\}$
- Cross: $\{A \rightarrow D, E \rightarrow H\}$)

c) Identify the strongly connected components and draw the component graph. [10 Points]

Using the graph reachability (or Kosaraju), the SCCs are:

1. **$\{B, A, E\}$** — cycle: $B \rightarrow E \rightarrow A \rightarrow B$ (all mutually reachable)
2. **$\{C\}$** — singleton (no return path to C)
3. **$\{F\}$** — singleton
4. **$\{H, D, G, I\}$** — cycle among these: $H \rightarrow G \rightarrow D \rightarrow H$ and $H \leftrightarrow I$ ($I \rightleftarrows H$), so all four are mutually reachable.

So the 4 SCCs (listing representative members):

- $S_0 = \{B, A, E\}$
- $S_1 = \{C\}$
- $S_2 = \{F\}$
- $S_3 = \{H, D, G, I\}$

6. A bipartite graph is a graph $G = (V, E)$ whose vertices can be partitioned into two sets ($V = V_1 \cup V_2$ and $V_1 \cap V_2 = \emptyset$) such that there are no edges between vertices in the same set (for instance, if $u, v \in V_1$, then there is no edge between u and v).

a) Design a linear-time algorithm to determine whether an undirected graph is bipartite. [10 points]

Submission not allowed afterwards

Total Marks: 100

BIPARTITE-CHECK(G):

for each vertex v in V :

$\text{color}[v] = \text{UNCOLORED}$

for each vertex s in V :

 if $\text{color}[s] == \text{UNCOLORED}$:

$\text{color}[s] = \text{RED}$

 create empty queue Q

 enqueue(Q, s)

 while Q is not empty:

$u = \text{dequeue}(Q)$

 for each neighbor w of u :

 if $\text{color}[w] == \text{UNCOLORED}$:

$\text{color}[w] = \text{opposite}(\text{color}[u])$

 enqueue(Q, w)

 else if $\text{color}[w] == \text{color}[u]$:

 return "Graph is NOT bipartite"

return "Graph IS bipartite"

b) At most how many colors are needed to color in an undirected graph with exactly one odd length cycle? [5 points]

2 colors

7. Use Prim's algorithm to compute a maximum spanning tree for the following graph shown in figure 2. Use node A as the root. You need to show each step. [10 Points]

b) Can Prim's and Kruskal's algorithm yield different maximum spanning trees? Explain why or why not. (5 points)

Yes

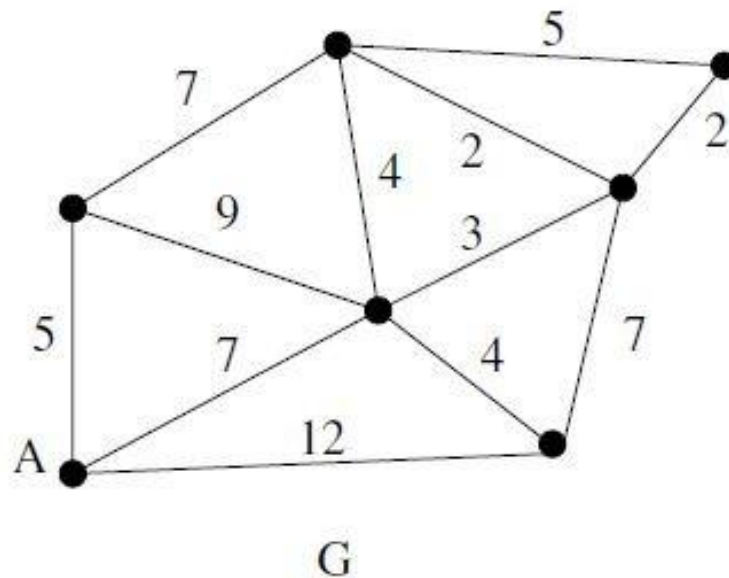


Figure 2: Graph for a maximum spanning tree

8. Using Floyd-Warshall, find all pairs shortest path for the given Figure 3 (D^0 weight matrix is also provided). Discuss its time and space complexity as well [10 Points]
9. Go through the lecture of Johnson's Algorithm part 1, Johnson's Algorithm part 2 and write its summary. [10 Points]
10. a) Give a simple example of a directed graph with negative-weight edges for which Dijkstra's algorithm produces incorrect answers.(5 points)
 Professor A suggests that to tackle negative-weight edges issue of Dijkstra's algorithm: add a large constant to each edge weight so that all the weights become positive, then run algorithm starting at node s , and return the shortest path found to node t .
 b) Is this a valid method? Either prove that it works correctly, or give a counterexample.(5 points)
 c) Consider a directed graph in which the only negative edges are those that leave s ; all other edges are positive. Can Dijkstra's algorithm, started at s , fail on such a graph? Prove your answer. [5 Points]
11. Read Book, Lectures or Search through Internet to explain the worst time complexities of BFS, DFS, Kruskal's and Prim's in your own words using (i) Adjacent Matrix (ii) Adjacent List [10 Points]

Due Date: 28 November 2025

20% penalty for 1 day late

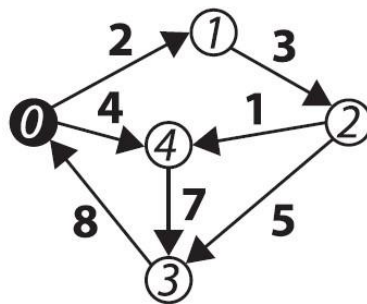
40% penalty for 2 days late

CS2009: Design and Analysis of Algorithms (Fall 2025)

Assignment 4

Submission not allowed afterwards

Total Marks: 100



	0	1	2	3	4
0	0	2	∞	∞	4
1	∞	0	3	∞	∞
2	∞	∞	0	5	1
3	8	∞	∞	0	∞
4	∞	∞	∞	7	0

Figure 3: find all pairs shortest path